

# Create your First Unit Test Method

Learn how to create effective unit tests that validate real code behavior with confidence and clarity.

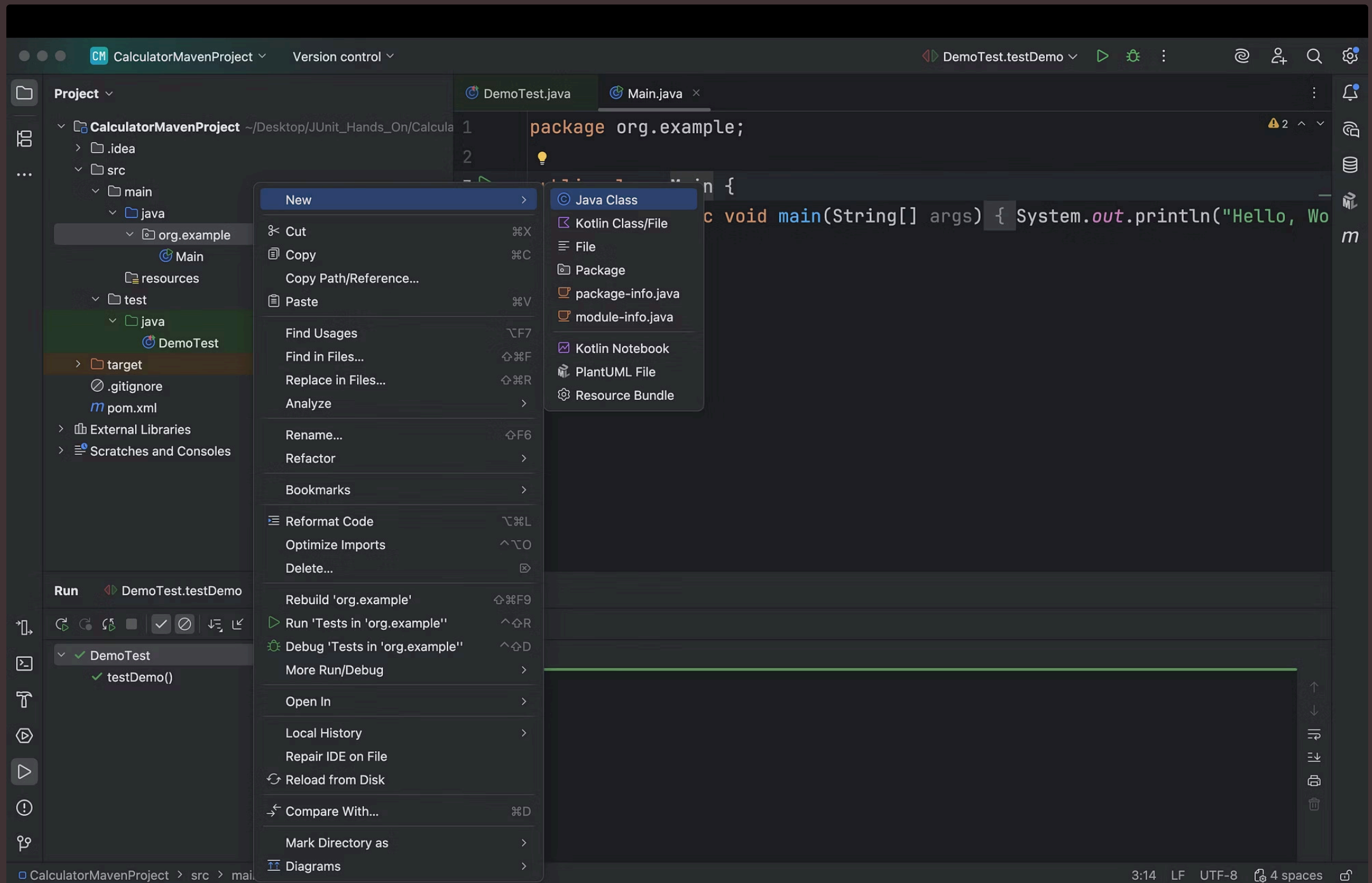
 by Rodrigo Martins Pagliares

Last update: 11/25/2025



# Creating a Class to be Tested

A.K.A System Under Test (SUT)



# Creating a Class to be Tested (contd.)

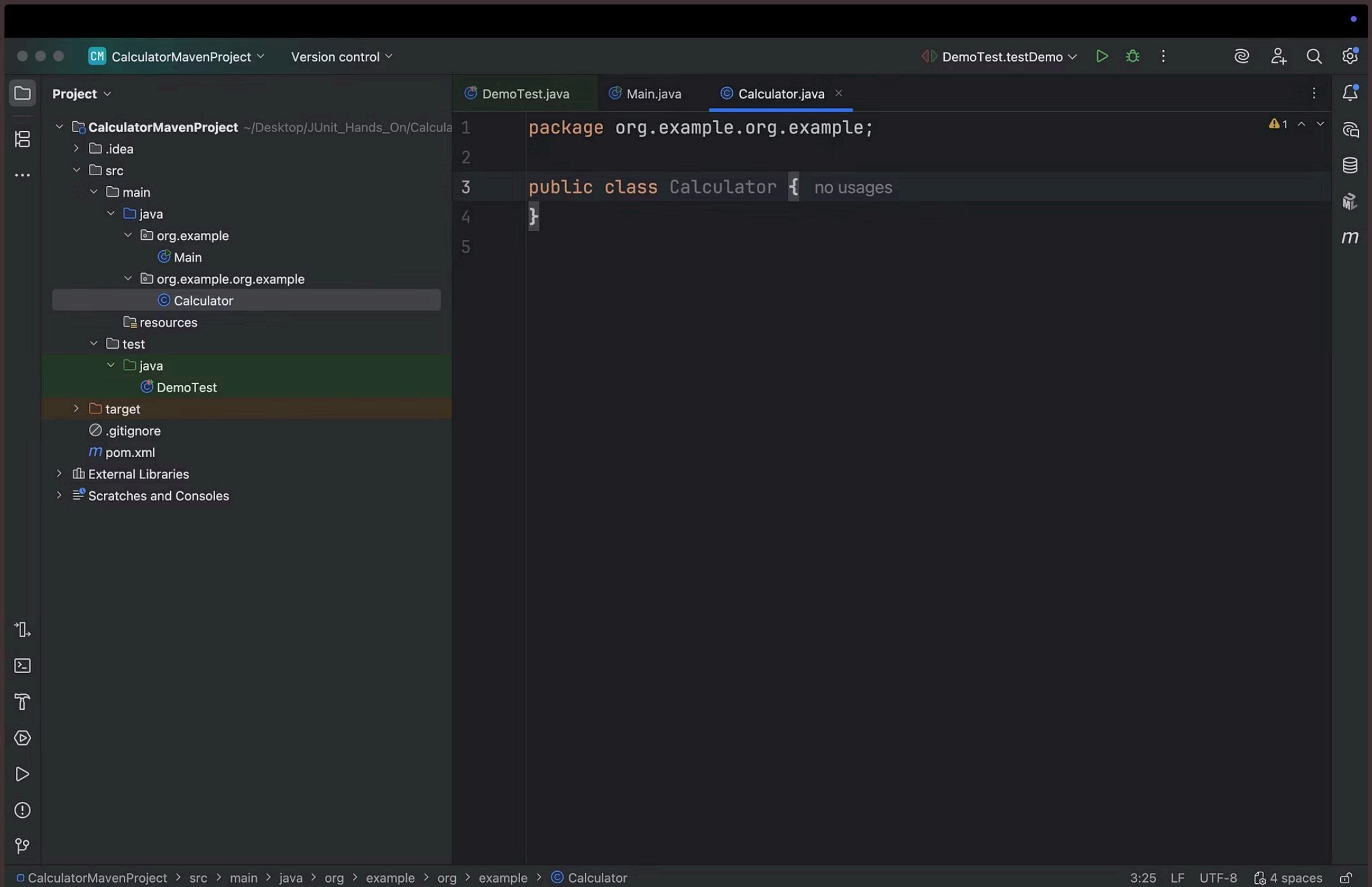
The screenshot shows an IDE window for a project named "CalculatorMavenProject". The left sidebar displays the project structure, with the "org.example" package under the "main" source folder selected. The main editor area shows the "Main.java" file with the following code:

```
1 package org.example;
2
3 public class Main {
4     public static void main(String[] args) { System.out.println("Hello, Wo
5
6
7 }
```

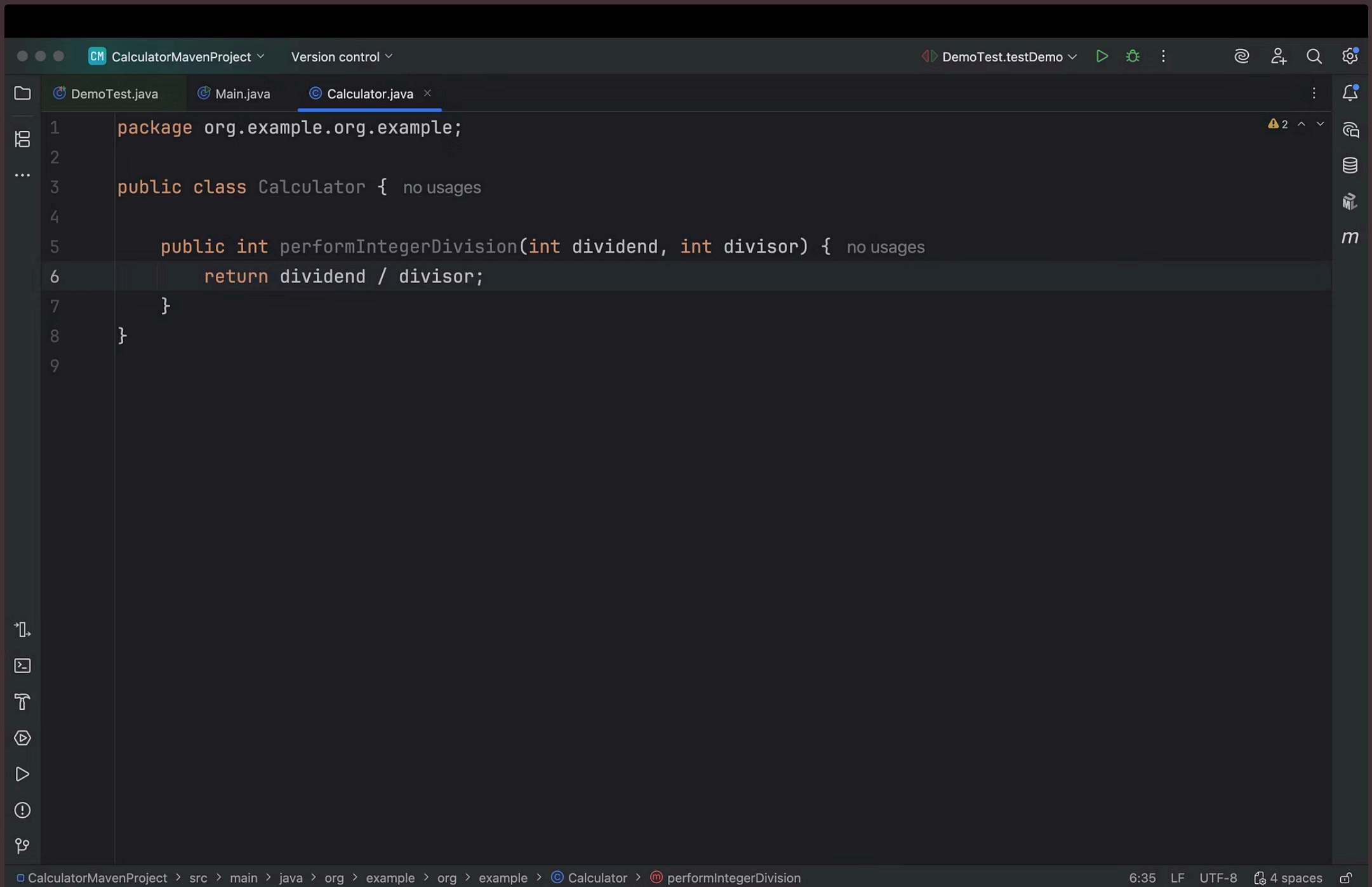
A "New Java Class" dialog box is open in the center, showing the package "org.example" and the class name "Calculator". The "Class" option is selected from the list of class types.

At the bottom, the "Run" tab shows the test results for "DemoTest.testDemo". The test passed successfully, taking 10 ms. The status bar at the bottom indicates the current file path is "CalculatorMavenProject > src > main > java > org > example" and the encoding is UTF-8.

# Creating a Class to be Tested (contd.)



# Creating a Class to be Tested (contd.)

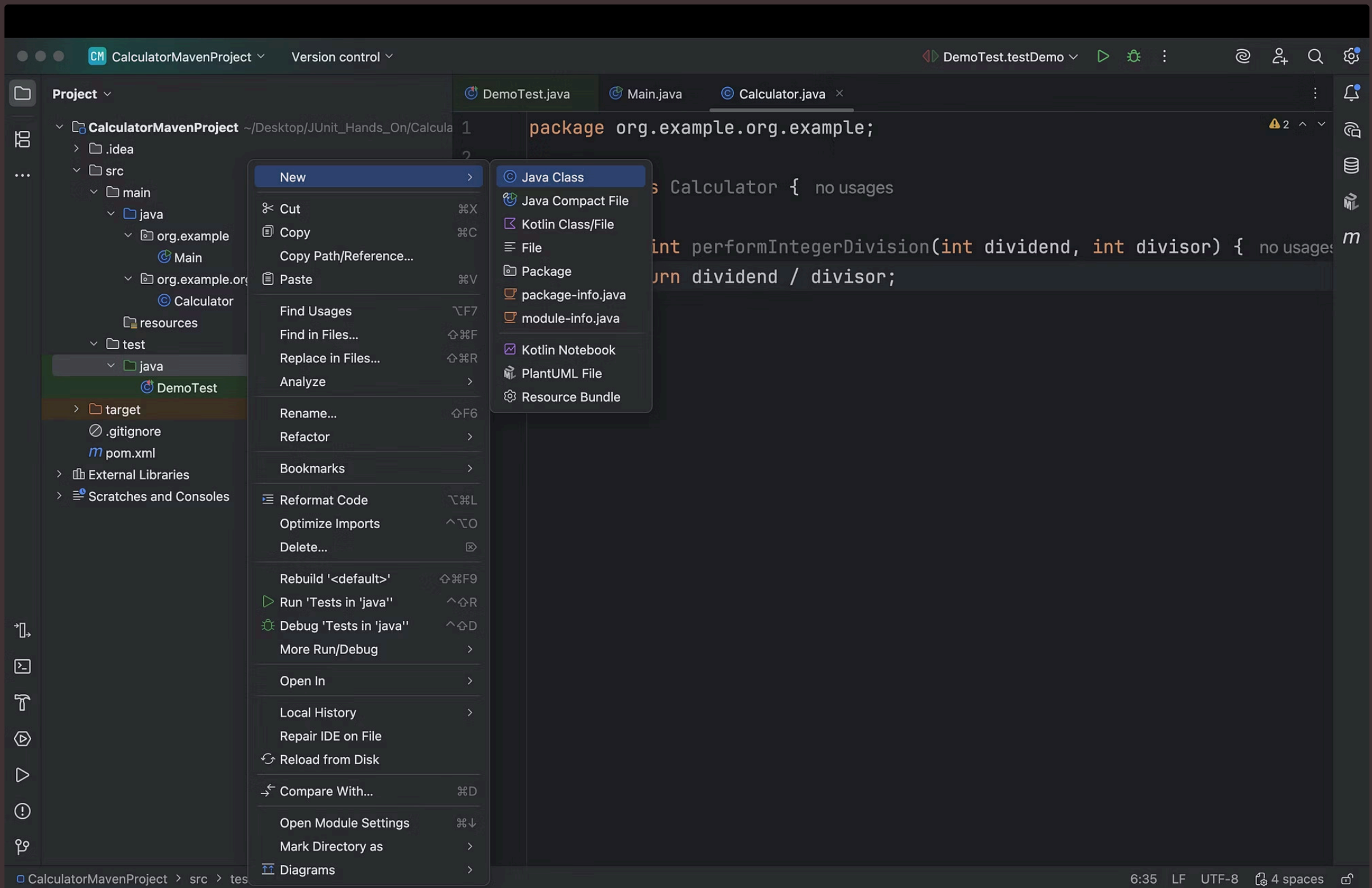


The screenshot shows an IDE window for a project named "CalculatorMavenProject". The "Version control" tab is active. The "DemoTest.testDemo" file is selected in the top right. The "Calculator.java" file is open in the editor, showing the following code:

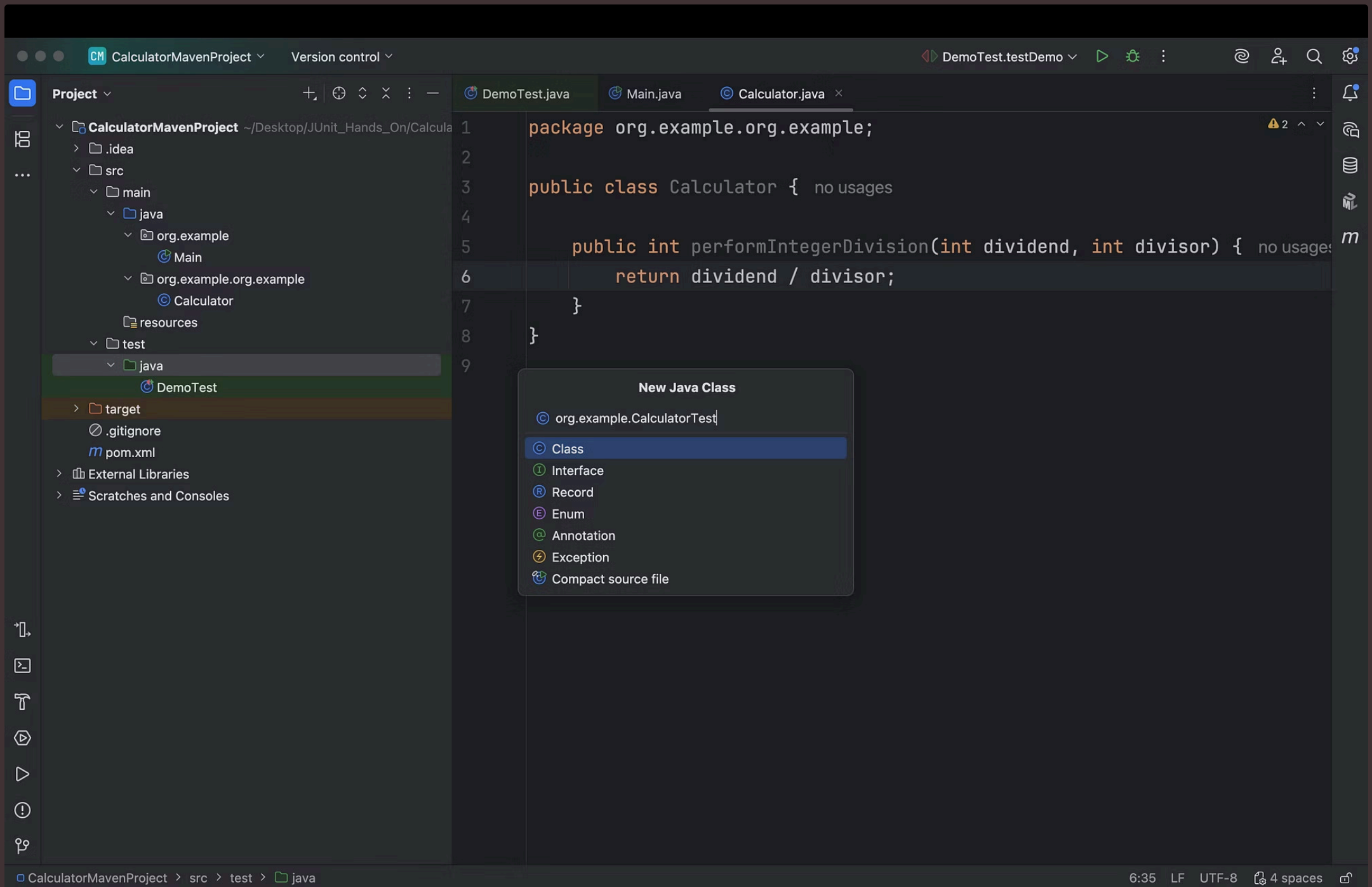
```
1 package org.example.org.example;  
2  
3 public class Calculator { no usages  
4  
5     public int performIntegerDivision(int dividend, int divisor) { no usages  
6         return dividend / divisor;  
7     }  
8 }  
9
```

The bottom status bar shows the file path: "CalculatorMavenProject > src > main > java > org > example > org > example > Calculator > performIntegerDivision". The status bar also displays "6:35", "LF", "UTF-8", and "4 spaces".

# Creating a Unit Test

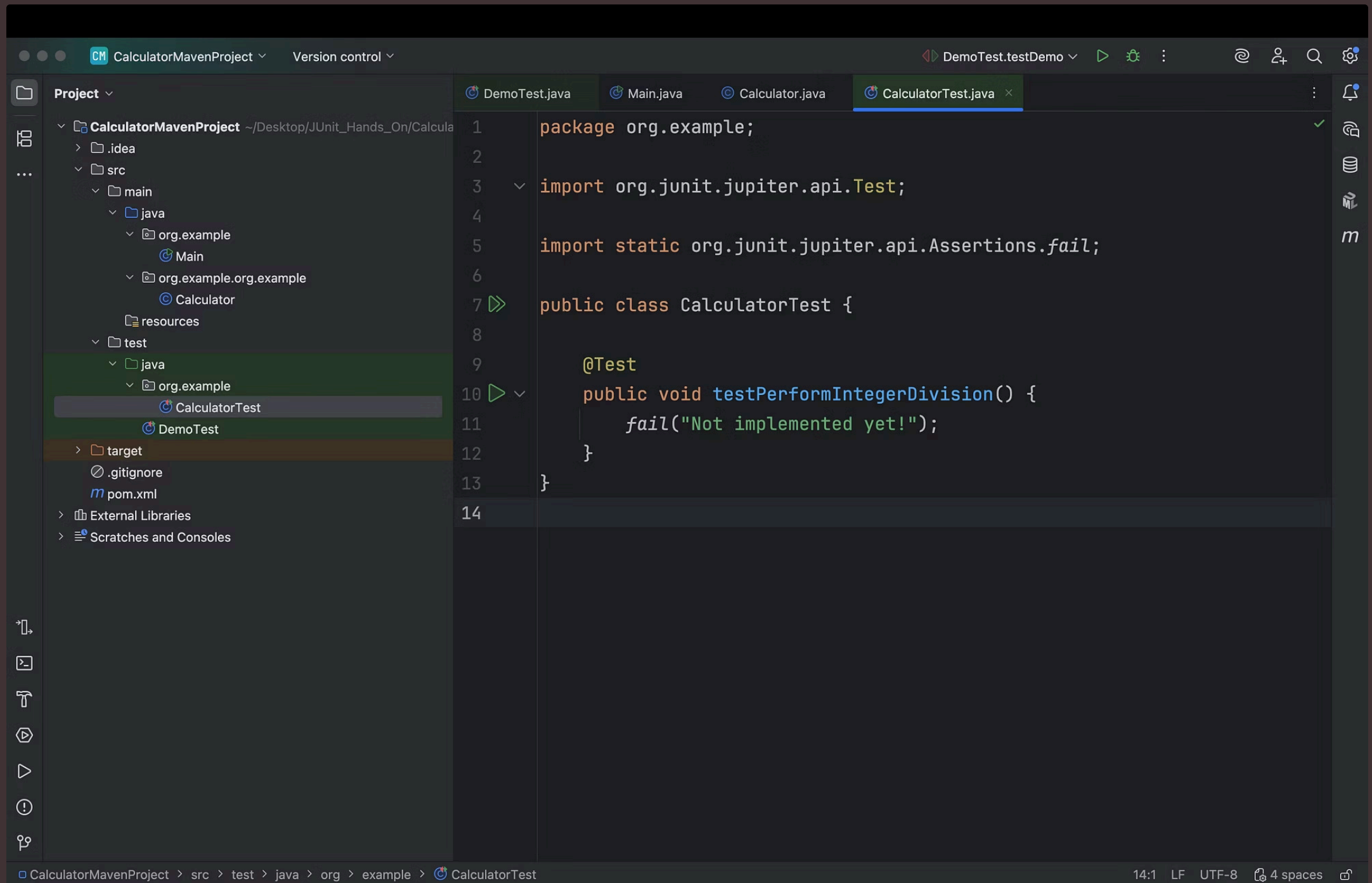


# Creating a Unit Test (contd.)



# Creating a Unit Test (contd.)

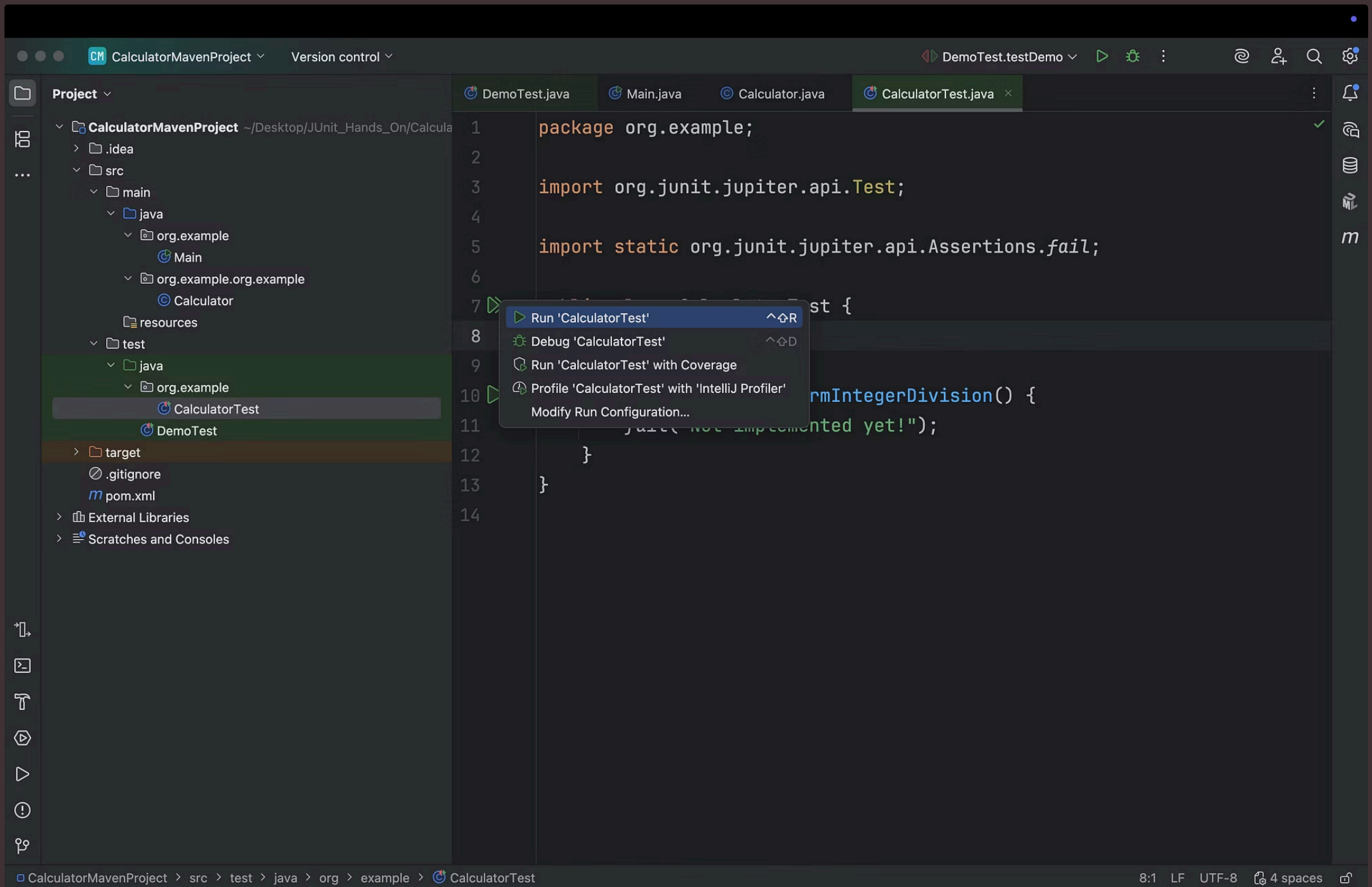
You can use the `fail` method to deliberately fail the test!



```
1 package org.example;
2
3 import org.junit.jupiter.api.Test;
4
5 import static org.junit.jupiter.api.Assertions.fail;
6
7 public class CalculatorTest {
8
9     @Test
10    public void testPerformIntegerDivision() {
11        fail("Not implemented yet!");
12    }
13 }
14
```

CalculatorMavenProject > src > test > java > org > example > CalculatorTest

# Running a Unit Test



# Running a Unit Test : Fail

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The left sidebar displays the project structure, with the "test" directory expanded to show the "org.example" package containing "CalculatorTest" and "DemoTest". The main editor displays the code for "CalculatorTest.java":

```
1 package org.example;
2
3 import org.junit.jupiter.api.Test;
4
5 import static org.junit.jupiter.api.Assertions.fail;
6
7 public class CalculatorTest {
8
9     @Test
10     public void testPerformIntegerDivision() {
11         fail("Not implemented yet!");
12     }
13 }
14
```

The "Run" toolbar at the bottom shows the test execution results: "1 test failed 1 test total, 15 ms". The test name is "testPerformIntegerDivision()". The error message displayed is:

```
org.opentest4j.AssertionFailedError: Not implemented yet!
> <2 internal lines>
at org.example.CalculatorTest.testPerformIntegerDivision(CalculatorTest.java:11)
```

The status bar at the bottom indicates the file path: "CalculatorMavenProject > src > test > java > org > example > CalculatorTest", the encoding: "UTF-8", and the line length: "8:1 LF UTF-8 4 spaces".

# Making the test pass: Building a Meaningful Unit Test

01

---

Replace placeholder assertions with real checks validating actual code behavior

02

---

Create an instance of the class containing the method under test (e.g., Calculator)

03

---

Call the target method with predefined inputs, such as `performIntegerDivision(10, 2)`

04

---

Store the returned value in a local variable for later comparison

05

---

Use assertions (e.g., `assertEquals`) to confirm the method returns the expected result

# Building a Meaningful Unit Test (contd.)

1. Remove the `fail` method,
2. Create an **instance** of the **class** containing the **method under test** (e.g., `Calculator`)
3. Call the **target method** with predefined **inputs**, such as `performIntegerDivision(10, 2)`
4. **Store the returned value** in a **local variable** for later comparison
5. Use **assertions** (e.g., `assertEquals`) to **confirm the method returns the expected result**

```
9 public class CalculatorTest {
10
11     @Test
12     public void testPerformIntegerDivision() {
13         Calculator calculator = new Calculator();
14         int result = calculator.performIntegerDivision( dividend: 10, divisor: 2);
15         assertEquals( expected: 5, result);
16     }
17 }
18
```

Run DemoTest.testDemo CalculatorTest

✓ CalculatorTest (org.example) 11ms  
✓ testPerformIntegerDivision() 11ms

✓ 1 test passed 1 test total, 11ms

/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...

Process finished with exit code 0

CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerDivision 15:32 LF UTF-8 4 spaces

# Deliberately Include a bug (+ instead of /)

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The editor displays the file "Calculator.java" with the following code:

```
1 package org.example.org.example;  
2  
3 public class Calculator { 3 usages  
4  
5     public int performIntegerDivision(int dividend, int divisor) { 1 usage  
6         return dividend + divisor;  
7     }  
8 }  
9
```

A lightbulb icon is visible next to line 6, indicating a potential issue. The "Run" tab at the bottom shows the execution of "CalculatorTest". The test results indicate that 1 test passed (1 test total, 11ms). The test output shows the command:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...
```

and the message:

```
Process finished with exit code 0
```

The bottom status bar shows the current file path: "CalculatorMavenProject > src > main > java > org > example > org > example > Calculator > performIntegerDivision". The status bar also displays the time "6:35", the file encoding "UTF-8", and the indentation "4 spaces".

# Test Failed!

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The file "CalculatorTest.java" is open, showing a test method `testPerformIntegerDivision()` that uses `assertEquals(5, result)`. The test failed because the actual result was 12 instead of the expected 5.

```
9 public class CalculatorTest {
10
11     @Test
12     public void testPerformIntegerDivision() {
13         Calculator calculator = new Calculator();
14         int result = calculator.performIntegerDivision(dividend: 10, divisor: 2);
15         assertEquals(expected: 5, result);
16     }
17 }
18
```

Run: DemoTest.testDemo x CalculatorTest.testPerformIntegerDivision x

CalculatorTest (org.example) 16 ms  
testPerformIntegerDivision() 16 ms

1 test failed 1 test total, 16 ms

/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...

org.opentest4j.AssertionFailedError:  
Expected :5  
Actual :12  
[Click to see difference](#)

> <6 internal lines>  
at org.example.CalculatorTest.testPerformIntegerDivision(CalculatorTest.java:15)

CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerDivision 15:32 LF UTF-8 4 spaces

# Fixing the code (/ instead of +)

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The editor displays the file "Calculator.java" with the following code:

```
1 package org.example.org.example;
2
3 public class Calculator { 3 usages
4
5     public int performIntegerDivision(int dividend, int divisor) { 1 usage
6         return dividend / divisor;|
7     }
8 }
9
```

Below the editor, the "Run" tab shows the test "CalculatorTest.testPerformIntegerDivision" failed. The error message is:

```
org.opentest4j.AssertionFailedError:
Expected :5
Actual   :12
<Click to see difference>

> <6 internal lines>
at org.example.CalculatorTest.testPerformIntegerDivision(CalculatorTest.java:15)
```

The bottom status bar indicates the file path: "CalculatorMavenProject > src > main > java > org > example > org > example > Calculator > performIntegerDivision". The status bar also shows "6:35 LF UTF-8 4 spaces".

# Test passed again!

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The "CalculatorTest.java" file is open, displaying the following code:

```
9 public class CalculatorTest {  
10  
11     @Test  
12     public void testPerformIntegerDivision() {  
13         Calculator calculator = new Calculator();  
14         int result = calculator.performIntegerDivision(dividend: 10, divisor: 2);  
15         assertEquals(expected: 5, result);  
16     }  
17 }  
18
```

Below the code editor, the "Run" tab is active, showing the execution of "CalculatorTest.testPerformIntegerDivision". The test results indicate that 1 test passed, taking 13 ms. The output window shows the command path and the message "Process finished with exit code 0".

Run: DemoTest.testDemo x CalculatorTest.testPerformIntegerDivision x

✓ CalculatorTest (org.example) 13 ms  
✓ testPerformIntegerDivision() 13 ms

✓ 1 test passed 1 test total, 13 ms

/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...

Process finished with exit code 0

CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerDivision 15:32 LF UTF-8 4 spaces

# Understanding **Test Failures**



- Changing business logic intentionally helps demonstrate how test failures are reported
- If the method's returned value differs from the **expected** one, the assertion fails
- **Failure** messages display the **expected** and **actual** values, aiding debugging
- Restoring the **correct business logic** causes the **test** to **pass** again
- **Green checkmarks** indicate **successful validation** in both **test** and **class** levels

# Complexity of Business Logic vs. Simplicity of Tests

Unit tests remain small and simple even when methods contain complex algorithms

You only need to verify input/output behavior, not internal implementation details

Tests focus on confirming that the method returns expected results for given inputs

Understanding all internal logic is optional—not required for writing effective tests

This mindset increases confidence and reduces fear of testing complex methods



# Open Floor Discussion

- Writing **effective unit tests** requires **clear inputs, expected outputs, and proper assertions**
- Even **complex methods** can be tested with **small, focused unit tests**
- **Failures** provide useful **feedback** and help **validate business logic correctness**
- Developing **confidence in testing** makes you more **productive** and **reduces regressions**
- What part of **writing your first real unit test** feels most **challenging**?
- Do you prefer **simple, input/output-focused tests** or more **detailed behavioral checks**?
- How confident do you feel **testing complex logic** after this explanation?

